

Backend Flow Explanation: database.py, meeting.py, and main.py

Big Picture Flow

When the backend starts:

1. database.py creates the database connection setup.
2. meeting.py defines the meetings table.
3. main.py creates the FastAPI app, creates the table if needed, and exposes API endpoints.

Then when a user uploads a file:

1. Request hits POST /api/meetings.
2. FastAPI receives the uploaded file and optional title.
3. Code validates the filename and extension.
4. File is saved into uploads/.
5. A row is created in the SQLite database with status queued.
6. API returns the new meeting_id.

1. database.py

This file is responsible for database connection plumbing.

```
from sqlalchemy import create_engine
```

Imports create_engine, which creates the main connection object SQLAlchemy uses to talk to the database.

```
from sqlalchemy.orm import sessionmaker, declarative_base
```

Imports two ORM tools:

- sessionmaker creates database sessions, which you use to query, insert, update, or delete rows.
- declarative_base creates a base class for your database models, like Meeting.

```
DATABASE_URL = "sqlite:///./app.db"
```

This says: use SQLite and store data in a local file called app.db. If you run the server from backend/, the file will usually be backend/app.db.

```
engine = create_engine(...)
```

The engine is SQLAlchemy's connection object. It knows how to talk to your SQLite database.

```
connect_args={"check_same_thread": False}
```

This is needed because FastAPI may handle requests across threads, and SQLite otherwise complains about thread usage.

```
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
```

This creates a session factory. A session is what your code uses to insert, query, update, or delete database records.

- autocommit=False means changes are not saved automatically; you must call db.commit().
- autoflush=False means SQLAlchemy will not automatically push pending changes before every query.
- bind=engine connects these sessions to your SQLite database engine.

```
Base = declarative_base()
```

This is the parent class your database models inherit from. Your Meeting model uses this.

```
def get_db():
```

This is a FastAPI dependency. Every time an API endpoint needs database access, FastAPI calls `get_db()`.

```
db = SessionLocal()
```

```
yield db
```

```
db.close()
```

It opens a database session, gives it to the endpoint, then closes it after the request finishes.

2. meeting.py

This file defines your database table.

```
class Meeting(Base):
```

This creates a SQLAlchemy model. A model is a Python class that maps to a database table.

```
__tablename__ = "meetings"
```

The table will be named meetings.

```
id = Column(Integer, primary_key=True, index=True)
```

Each meeting gets a unique integer ID. This is the `meeting_id` your API returns.

```
title = Column(String, nullable=True)
```

The meeting title is optional. If the user does not provide one, it can be NULL.

```
filename = Column(String, nullable=False)
```

The saved upload filename is required. This stores the generated safe filename, not necessarily the original user filename.

```
status = Column(String, nullable=False, default="queued")
```

The meeting processing status starts as queued. Right now your planned statuses are queued, processing, done, and failed. Currently the code only creates queued; no worker exists yet to update it.

```
created_at = Column(DateTime(timezone=True), server_default=func.now())
```

The database automatically records when the meeting row was created.

3. main.py

This is the actual FastAPI application.

```
UPLOAD_DIR = "uploads"
```

```
os.makedirs(UPLOAD_DIR, exist_ok=True)
```

This says uploaded files should go into an uploads folder. If that folder does not exist, create it. Because this is a relative path, it depends where you run the server from.

```
app = FastAPI(title="Meeting Assistant API")
```

Creates the FastAPI app.

```
Base.metadata.create_all(bind=engine)
```

This tells SQLAlchemy to create any missing database tables based on models like Meeting. If app.db does not exist yet, this helps create the meetings table.

```
@app.get("/health")
```

```
def health():
```

```
    return {"ok": True}
```

A simple health check endpoint. If you visit /health, it returns {"ok": true}. This is useful to confirm the backend is running.

Upload Endpoint

```
@app.post("/api/meetings")
```

```
async def create_meeting(...)
```

This handles file uploads.

```
file: UploadFile = File(...)
```

This means a file is required in the request.

```
title: str | None = Form(default=None)
```

This means the user may optionally send a title as form data.

```
db: Session = Depends(get_db)
```

This tells FastAPI to give this endpoint a database session from get_db().

```
if not file.filename:
```

```
    raise HTTPException(status_code=400, detail="Missing filename")
```

If the uploaded file has no filename, reject the request with 400 Bad Request.

```
ext = os.path.splitext(file.filename)[1].lower()
```

Extracts the file extension, like .mp3 or .mp4.

```
allowed = {".mp3", ".wav", ".m4a", ".mp4", ".webm"}
```

Only these audio/video file types are allowed.

```
if ext not in allowed:
```

```
    raise HTTPException(...)
```

Reject unsupported files.

```
safe_name = f"{uuid.uuid4().hex}{ext}"
```

Creates a new random filename. This avoids problems with strange user filenames and prevents overwriting files with the same name.

```
save_path = os.path.join(UPLOAD_DIR, safe_name)
```

Builds the final upload path.

```
contents = await file.read()
```

Reads the uploaded file into memory. For small MVP files this is okay. Later, for big video files, streaming or chunked saving is better.

```
with open(save_path, "wb") as f:
```

```
    f.write(contents)
```

Saves the uploaded file to disk.

```
meeting = Meeting(title=title, filename=safe_name, status="queued")
```

Creates a new database object in Python.

```
db.add(meeting)
```

```
db.commit()
```

```
db.refresh(meeting)
```

This inserts the row into the database, saves it, and refreshes the object so meeting.id is available.

```
return {"meeting_id": meeting.id, "status": meeting.status}
```

Returns the ID and current status to the frontend.

Status Endpoint

```
@app.get("/api/meetings/{meeting_id}/status")
```

This lets the frontend ask: Is this meeting done yet?

```
meeting = db.query(Meeting).filter(Meeting.id == meeting_id).first()
```

Looks for a meeting row with that ID.

```
if not meeting:
```

```
    raise HTTPException(status_code=404, detail="Meeting not found")
```

If no meeting exists, return 404.

```
return {"meeting_id": meeting.id, "status": meeting.status}
```

Returns the meeting status.

Meeting Detail Endpoint

```
@app.get("/api/meetings/{meeting_id}")
```

This gets basic metadata about one meeting.

It returns meeting_id, title, filename, status, and created_at.

Current System Flow

1. You start the backend with `uvicorn app:app --reload --port 8000`.
2. FastAPI starts.
3. `main.py` imports `Base`, `engine`, and `get_db` from `database.py`.
4. `main.py` imports `Meeting` from `meeting.py`.
5. `Base.metadata.create_all(bind=engine)` creates the meetings table if missing.
6. User uploads a file to `POST /api/meetings`.
7. The uploaded file gets saved in `uploads/`.

8. A database row is inserted into meetings.
9. The API returns the new meeting_id.
10. Frontend can poll GET /api/meetings/1/status.
11. Backend looks up meeting 1 and returns its status.

Current Limitation

Right now this is not doing transcription yet. It only does:
upload file -> save file -> create DB row -> return queued status

That is still exactly what Step 1 should do. The next step would be adding a background worker or temporary fake processing function that changes queued -> processing -> done. Then later you connect Whisper to actually transcribe the saved upload.